KGRAG: A Knowledge Compiler*and Federated Retrieval Layer for Ontologically Grounded Domains

Eric G. Suchanek, PhD
Flux-Frontiers, Liberty Township OH

https://github.com/Flux-Frontiers/KGRAG

Abstract

KGRAG is a federation and orchestration layer for structural knowledge graphs derived from heterogeneous source domains: Python source code (CodeKG), natural language documentation (DocKG), scientific or domain-specific corpora (MetaboKG), and a growing set of formally structured domains including biochemical pathway data, protein structure files, legal codes, personal diary corpora, scriptural texts, and biographical knowledge graphs. Each backend constructs its knowledge graph from the formal structure of its source—abstract syntax trees for code, document section and reference graphs for text, domain ontologies for scientific data—without any language model participation in graph construction.

The central thesis of this work is that the power of structural knowledge-graph retrieval scales directly with the formal completeness of the source domain's ontology. Programming languages represent the ideal case: their grammars are complete formal specifications, and the derived call graph is correct by theorem. The same principle applies to biochemical pathway schemas, statutory hierarchies in books of law, protein data bank file formats, and the structured hierarchies of scripture and diary. KGRAG treats all of these domains identically through a uniform five-method adapter protocol.

The design treats the structurally derived graph as ground truth and uses semantic embeddings strictly as an acceleration layer for locating entry points into that graph. Graph traversal, ranking, and snippet extraction are deterministic: identical inputs produce identical outputs, and no component of the structural pipeline produces probabilistic results. When KGRAG output is passed to a large language model for synthesis, the model receives verified facts with full source provenance, not approximate embeddings. Hallucination risk is bounded to the synthesis stage and is zero at the knowledge-retrieval stage.

An MCP server exposes the full federated query surface—including corpus-scoped and person-scoped operations—to any MCP-compatible AI agent. The long-term goal of this architecture is the *TreeOfKnowledge™*, also termed the *Corpus Scientia*: all human knowledge, all domains, one interface, one query, coupled with the largest available language model for grounded synthesis.

1 Introduction

Large software systems accumulate knowledge in incompatible forms. Source code encodes behavior and architecture in a machine-readable but semantically dense notation. Documentation encodes intent, rationale, and usage patterns in prose. Domain corpora—pathway databases, ontologies, financial models, legal codes, scripture—encode facts in formats specific to each field. These representations are maintained separately, queried separately, and surfaced separately; an analyst or an AI agent that needs to understand a system must traverse all of them and reconcile the results manually.

*The *Knowledge Compiler* concept and its execution are the subject of a pending patent application (U.S. provisional filed). Patent Pending.

The standard answer is retrieval-augmented generation (RAG): embed all documents in a vector index and retrieve the closest matches to a query. This approach treats source code, scientific data, and legal text as undifferentiated text, discarding the formal structure that makes those artifacts tractable. A Python call graph cannot be derived from a vector search; neither can a metabolic pathway traversal, a statutory cross-reference chain, or a protein bond topology. Vector retrieval approximates relevance; it does not expose structure.

An alternative, exemplified by GraphRAG [1], is to use a language model to extract a knowledge graph from unstructured text. This recovers some structure, but the structure is inferred rather than derived: entity extraction is probabilistic, edges may be hallucinated, and the resulting graph cannot be independently verified against any formal source.

KGRAG takes a different approach, grounded in a fundamental observation about knowledge domains.

1.1 The Ontology Principle

The power of structural knowledge-graph retrieval scales directly with how well-defined the ontology of the source domain is. A *well-defined ontology* is one where the entities, their relationships, and the rules governing those relationships are formally specified—not inferred, not approximated, but written down and enforced by construction.

Programming languages represent the ideal case. Python's grammar is a complete formal specification; every function, class, import, and call site is expressed in that grammar; and a deterministic parser can extract every structural relationship without any inference. The derived call graph is not an approximation—it is a theorem about the source code. The same property holds, to varying degrees, across a surprising breadth of domains:

Domain	Formal structure	KG kind
Python source code	Abstract syntax tree	`code`
Markdown / RST documentation	Document parse tree, hyperlinks	`doc`
Biochemical pathways	Reaction schemas (KEGG, BioCyc)	`meta`
Protein structures	PDB file format (ATOM, SEQRES)	`pdbfile`
Disulfide bond networks	Bond topology from PDB coordinates	`disulfide`
Books of law	Title → Chapter → Section → Clause	`legal`
Diary / journal entries	Document structure + temporal order	`diary`
Scripture / verse	Book → Chapter → Verse, cross-refs	`verse`
Episodic memory	Temporal event graphs	`memory`
Personal knowledge	Biographical and relational graphs	`person`
Conversational memory	Turn/topic/task graph (live session)	`agent`
File system tree	Directory/file/module/dependency graph	`filetree`

In every case, the formal structure of the source *is* the ontology, and a knowledge graph derived from it carries the same correctness guarantees as the source itself. This is the foundational principle of KGRAG: build knowledge graphs from formal structure, and the graphs are correct by construction.

1.2 Contributions

The contribution of this paper is not a new graph construction algorithm but a new level of abstraction above existing single-domain tools: a uniform federation layer, termed a *knowledge compiler*, that makes heterogeneous structural knowledge graphs queryable as a single unified corpus. Specifically:

1. A five-method adapter protocol (`KGAdapter`) that decouples the federation layer from all backend-specific logic, enabling new knowledge domains to be added by implementing five methods with no changes to the orchestrator, registry, CLI, or MCP server.
2. A corpus abstraction grouping multiple KG instances for scoped federated queries, and a person corpus abstraction extending this with personal metadata.
3. A complete MCP server exposing registry, corpus, and person tools to AI agents.
4. A demonstration that structural KG retrieval followed by language model synthesis eliminates hallucination at the knowledge layer, bounding risk to the synthesis stage alone.
5. A snapshot system enabling *differential queries*: what changed between two compiled states of a knowledge graph, and what new knowledge entered a corpus over a given interval.
6. An incremental ingestion pipeline with diversity-preserving temporal sampling and resumable state, enabling progressive compilation of large corpora without blocking queries.
7. A long-term architectural roadmap toward the *TreeOfKnowledge*™ (*Corpus Scientia*): all human knowledge, all domains, one federated interface.

2 Design Principles

Six principles govern the design of KGRAG and its backend modules.

1. **Derived structure is authoritative.** Knowledge graphs are extracted from formal sources—ASTs, document parse trees, ontological schemas—by deterministic programs. The graph is not a model of the source; it *is* the source, represented relationally. Embeddings are derived from it and are disposable.
2. **Semantics accelerate; structure decides.** Vector search locates entry points. Graph traversal determines what is returned. The two phases are explicit, separable, and independently tunable. A change to the embedding model affects which seeds are selected; a change to the hop count or edge filter affects traversal depth. Neither affects the structural layer.
3. **Every result is traceable.** Every node carries a stable identifier encoding its origin: file path and qualified name for code nodes, document path and section offset for documentation nodes. Every snippet is extracted from a verified location. Nothing enters a result without a source address.
4. **Determinism over approximation.** Given identical inputs, the system produces identical outputs. Ranking is computed by a deterministic composite key. There are no stochastic components in the structural or traversal layers.
5. **Generality through protocol.** The federation layer depends on an abstract adapter interface, not on any specific backend. Adding a new knowledge domain requires implementing five methods; the orchestrator, registry, CLI, and MCP server require no changes.
6. **Independence from language models.** The build pipeline, query engine, graph traversal, ranking, snippet extraction, and analysis tools all run locally without any language model call. A downstream language model may consume the output, but it plays no role in producing it.

3 Architecture Overview

The system is organized into five abstraction layers, each with a single responsibility and depending only on layers below it. Figure 1 shows their arrangement.

Layers 1–3 are implemented independently by each knowledge graph module and are hidden behind the Layer 4 boundary. Layer 5 operates exclusively through the `KGAdapter` interface and has no knowledge of any module's internal representation.

Layer 5: Federation
`KGRAG` orchestrator, `KGRegistry`, `CorpusRegistry`, `PersonCorpusRegistry`, MCP server
Layer 4: Adapter Protocol
`KGAdapter`: `is_available`, `query`, `pack`, `stats`, `analyze`
Layer 3: Semantic Indexing
LanceDB vector store, sentence-transformer embeddings
Layer 2: Graph Storage
SQLite, typed nodes and edges, BFS traversal, provenance
Layer 1: Source Parsing
AST (code), document parse tree (docs), ontological schema (domain)

Figure 1: KGRAG abstraction layers. Each layer depends only on those below it. The adapter protocol at Layer 4 is the boundary between domain-specific and domain-agnostic components.

4 Layer 1: Source Parsing

Each knowledge graph module derives its graph from the formal structure of its source domain. Six full adapters are currently implemented—code (PyCodeKG), documentation (DocKG), metabolic pathways (MetaboKG), diary corpora (DiaryKG), conversational memory (AgentKG), and file system trees (FTreeKG)—with stub adapters providing the protocol boundary for six additional domains under active development: `pdbfile`, `disulfide`, `legal`, `verse`, `memory`, and `person`.

4.1 Python Codebase Parsing (PyCodeKG)

`PyCodeKG.extract()` performs three deterministic AST passes over all `.py` files in a repository. Pass 1 extracts definitions and emits structural edges: `CONTAINS` (module to class, class to method), `IMPORTS` (module-level import relationships), and `INHERITS` (class to base class). Pass 2 extracts the call graph, emitting `CALLS` edges annotated with call-site evidence: the line number and expression text of each call site, extracted directly from the AST node.

Pass 3 runs a data-flow visitor (`PyCodeKGVisitor`) that extends `ast.NodeVisitor` to emit three additional edge types per function: `READS` (variable consumption), `WRITES` (variable assignment), and `ATTR_ACCESS` (attribute access on a named object). These capture intra-function data movement without requiring execution or type inference.

A post-build step, `resolve_symbols()`, name-matches every unresolved call target (represented as a `sym:` stub node) against all first-party definitions and writes `RESOLVES_TO` edges. This enables cross-module fan-in queries: given a function, which functions in other modules call it?

All nine declared edge relations—`CONTAINS`, `IMPORTS`, `INHERITS`, `CALLS`, `READS`, `WRITES`, `ATTR_ACCESS`, `DEPENDS_ON`, and `RESOLVES_TO`—are derived from syntax. No relation is inferred.

4.2 Document Corpus Parsing (DocKG)

DocKG parses Markdown corpora into a heterogeneous graph. Six node kinds are extracted: `document`, `section`, `chunk`, `entity`, `keyword`, and `topic`. Eight edge types connect them. Structural edges follow the document hierarchy; semantic edges are derived from co-occurrence and overlap analysis without language model involvement.

4.3 Domain Corpus Parsing (MetaboKG and Beyond)

MetaboKG wraps a domain-specific backend—initially targeting metabolic pathway databases—via the same `KGAdapter` interface. DiaryKG parses dated diary entries (plain text or PDF) into timestamped chunk graphs with temporal edges. AgentKG builds a live conversational memory graph from session turn sequences, emitting Turn, Topic, Task, and Summary nodes that are queryable across sessions. FTreeKG derives a structural graph of a file system repository: directories, files, modules, and their dependency and import relationships, enabling layout and architecture queries over any codebase or document tree. The parsing strategy of each backend is domain-defined; the interface contract is uniform. A biochemical pathway graph derived from KEGG, a diary corpus derived from plain-text journal files, a conversational session derived from turn sequences, and—once the corresponding compilers are complete—a protein topology derived from PDB ATOM records, a legal code derived from statutory filings, and a verse graph derived from scripture chapter/verse structure will all surface identically to the federation layer.

This uniformity is not accidental. It is the central claim of the ontology principle: *where formal structure exists, structural parsing produces graphs with the same correctness guarantees as the source, and the federation layer need not know which domain it is querying.*

5 Layer 2: Graph Storage

All modules persist their graphs to SQLite in a two-table schema: a `nodes` table and an `edges` table. Node records carry a stable string identifier, a kind, a name, and a JSON metadata blob. Edge records carry source node ID, relation name, destination node ID, and optional JSON evidence.

Node identifiers are deterministically constructed from kind, source path, and qualified name. For example: `m:src/kg_rag/orchestrator.py:KGRAG.query` (methods use the `m:` prefix; module-level functions use `fn:`). This means a node ID is stable across rebuilds as long as the source does not move, and it is human-readable and directly linkable to the source.

The key traversal primitive is `expand(seed_ids, hop, rels)`: a bounded breadth-first search from a set of seed nodes, following a specified set of edge relations up to a configurable hop limit. Each reached node is annotated with `ProvMeta`: its minimum hop distance from any seed and the originating seed node ID. Provenance travels with every result.

6 Layer 3: Semantic Indexing

`SemanticIndex` reads nodes from a `GraphStore`, constructs a canonical text document for each node (name plus available descriptive text), embeds them using a sentence-transformer model, and stores the vectors in LanceDB. The index is derived from the SQLite graph and is fully disposable. Its only function is to rank nodes by semantic similarity to a query string, producing a short list of seed node IDs for the subsequent structural traversal.

The separation is intentional and load-bearing. If the embedding model is replaced, the new index will select different seeds; the traversal from those seeds will still follow the same structural edges and produce structurally correct results. The accuracy of the structural layer does not depend on the quality of the embedding model.

6.1 The Compilation Cost: Paid Once

Embedding is the computationally expensive phase of the KGRAG pipeline, and it is worth making this explicit. Encoding tens of thousands of nodes into high-dimensional vectors using a sentence-transformer

model is a non-trivial compute task; for a large diary corpus or a full legal code, it may require minutes to hours on CPU hardware.

This is not a deficiency. It is an instance of a well-understood engineering trade-off: the compiler trade-off. Compiling a large Fortran simulation suite may take tens of minutes, but the resulting binary executes in milliseconds and may be run any number of times without recompilation. The compile cost is amortised across every subsequent execution.

KGRAG applies the same trade-off. The *build* step—parse, extract, store, embed—is the compilation. Each subsequent *query* is execution: instantaneous, deterministic, and repeatable. A federated query over a codebase of ∼100 k lines runs in under one second. The embedding cost was paid during build, once, and is not paid again until the corpus changes.

This framing also clarifies what the compiled artefact represents. A compiled binary is the source program, in executable form. The compiled knowledge graph—SQLite nodes and edges plus LanceDB vectors—*is* the corpus, in queryable form. Just as one does not re-read the source to run the program, one does not re-parse the corpus to answer a query. The structure is already there, addressable in constant time.

7 Layer 4: The Adapter Protocol

`KGAdapter` is an abstract base class with five abstract methods. Every knowledge graph module exposes itself to the federation layer through this interface and through no other.

```
class KGAdapter(ABC):
    def __init__(self, entry: KGEntry, embedder=None) -> None: ...

    @abstractmethod
    def is_available(self) -> bool: ...

    @abstractmethod
    def query(self, q: str, k: int = 8,
              min_score: float = 0.0,
              semantic_floor: float = 0.0) -> list[CrossHit]: ...

    @abstractmethod
    def pack(self, q: str, k: int = 8,
             context: int = 5,
             semantic_floor: float = 0.0) -> list[CrossSnippet]: ...

    @abstractmethod
    def stats(self) -> dict[str, Any]: ...

    @abstractmethod
    def analyze(self) -> str: ...

    # Concrete template method -- not abstract; subclasses override
    # _collect_snapshot_metrics() to add domain-specific fields.
    def snapshot(self, version: str,
                 label: str | None = None) -> dict[str, Any]: ...

    # Internal -- strips domain-specific fields; used by the orchestrator
    # to aggregate graph-size metrics uniformly across all KG kinds.
    def _graph_stats(self) -> dict[str, int]: ...
```

The `snapshot` method is a *concrete template method* on the base class: it builds a standard envelope (version string, ISO 8601 UTC timestamp, node count, edge count) and merges in any domain-specific data returned by the subclass hook `_collect_snapshot_metrics()`. Subclasses extend the snapshot by overriding only that hook, not `snapshot` itself. The `_graph_stats` helper is likewise concrete: it strips all domain-specific keys from `stats()` and normalises counts to integers so the orchestrator can aggregate graph-size totals uniformly across heterogeneous KG kinds without being confused by domain-specific fields such as `reaction_count` or `chunk_count`.

A `StubKGAdapter` base class provides a graceful default implementation for domains whose backing library has not yet been released. Stub adapters return empty results from `query` and `pack`, report `status: unavailable` from `stats`, and produce a structured unavailability notice from `analyze`. This allows new domains to be registered, grouped into corpora, and included in federated queries before their compilers are complete; they participate in the structure of the knowledge graph but contribute no data until ready.

The adapter protocol defines the boundary between the universal and the specific. Everything above Layer 4 is domain-agnostic.

8 Layer 5: Federation

8.1 Registry

`KGRegistry` maintains a persistent SQLite table of `KGEntry` records. Each entry records the KG instance name, its kind, the repository root, paths to its SQLite and LanceDB databases, the module version, and a JSON metadata blob. Registration is performed once per project; subsequent queries consult the registry to locate all known KG instances.

8.2 Corpus and Person Corpus Registries

Two higher-level registries group KG instances for scoped federated queries.

`CorpusRegistry` maintains a `corpora` table in the same SQLite file. A `CorpusEntry` is a named list of `KGEntry` UUIDs with an optional description and tags. Corpus operations (`query_corpus`, `pack_corpus`, `stats_corpus`) resolve the UUID list to `KGEntry` objects and dispatch to the same adapter machinery as global queries.

`PersonCorpusRegistry` extends this with personal metadata: birth year, birth date, address, email, phone, and free-form notes. A `PersonCorpusEntry` is the natural grouping for all KGs relevant to an individual—diary entries, memory graphs, documents, code, verse, correspondence. Person queries fan out over all of the person's KGs:

```
kgrag corpus person query "Eric Suchanek" \
    "disulfide bond research 2019"
```

8.3 Orchestrator

`KGRAG` is the cross-KG orchestrator. On each query, it:

1. Retrieves all entries from the registry, optionally filtered by kind.
2. For each entry, instantiates (or retrieves from cache) the corresponding `KGAdapter` and verifies availability.
3. Dispatches the query string to each available adapter in sequence, collecting `CrossHit` or `CrossSnippet` lists.

4. Merges all results into a single list and sorts by score descending.
5. Returns a typed result object with the merged list and per-KG metadata.

The dispatch loop is deliberately simple. Individual adapter calls are independent; a failure in one does not abort the others. Stub adapters (for domains whose libraries are not yet available) return empty lists and are silently skipped.

8.4 Result Types

`CrossQueryResult` carries the merged hit list, the per-KG breakdown, the total hit count, and the number of KGs that were actually queried.

`CrossSnippetPack` carries the merged snippet list and an approximate token count for budget-aware context assembly. `CrossSnippetPack.render()` produces a single LLM-ready string with section headers identifying each snippet's source domain, KG name, file path, and line span.

9 Query Execution

A federated query executes as follows at each participating adapter:

1. **Semantic seeding.** The query string is embedded and used to retrieve k seed nodes from LanceDB.
2. **Structural expansion.** `GraphStore.expand()` performs BFS from the seed IDs, following a specified set of edge relations up to a configurable hop limit. Each reached node is annotated with `ProvMeta`.
3. **Ranking.** Nodes are ranked by a deterministic composite key: hop distance (ascending), seed embedding distance (ascending), node kind priority, node ID as tiebreaker.
4. **Deduplication.** For snippet queries, nodes are deduplicated by file and source span.
5. **Snippet extraction.** Source text is extracted at each retained node's verified path and line span, with a configurable context window.

After each adapter completes, the orchestrator merges results and performs a final global sort by score. End-to-end latency for a federated query over a codebase of $\sim$100 k lines is typically under one second on commodity hardware.

10 Grounded Synthesis and the Hallucination Bound

When the output of a KGRAG federated query is passed to a large language model for synthesis, the combination constitutes what we term *Structural KG-RAG* [2]: knowledge-graph-augmented generation in which the knowledge graph was not itself produced by a language model.

Every piece of context in the synthesis prompt carries:

- A stable node identifier encoding its source address
- A source file path and line span
- A deterministically computed relevance score
- The actual source text

The language model is presented with *facts*, not *embeddings*. Its role is synthesis—connecting, explaining, summarizing—not construction of the knowledge itself. The critical consequence is that **hallucination risk at the knowledge-retrieval stage is zero.** The model cannot fabricate a function call that is not in the graph, because the graph was derived from the AST; once LegalKG is compiled, it will not be able to invent a statute cross-reference absent from that graph, because the graph will be derived from the formal text; it cannot hallucinate a protein bond that is not in the PDB file, because the bond graph was derived from atomic coordinates.

Hallucination risk is confined to the synthesis stage alone—where the model may reason incorrectly over correct facts. This risk is substantially lower and is addressed by standard techniques (chain-of-thought, self-consistency, re-querying). The knowledge foundation is provably correct by construction.

This is the decisive advantage of structural KG-RAG over inference-based approaches: the correctness guarantee is unconditional with respect to the structural layer and does not depend on prompt engineering, model scale, or retrieval quality.

11 The TreeOfKnowledge™: Corpus Scientia

The architecture described in this paper is a foundation for a larger goal: the *TreeOfKnowledge™*, also termed the *Corpus Scientia*.

> *All human knowledge. All domains. One interface. One query.*

The ontology principle establishes that any domain with well-defined formal structure can be compiled into a knowledge graph using the KGRAG adapter protocol. The federation layer then makes all such graphs queryable as a single unified corpus. As domain-specific compilers (DiaryKG, LegalKG, DisulfideKG, PDBFileKG, VerseKG, MemoryKG, PersonKG) are built and registered, the system progressively approaches completeness across domains:

- **Code corpora** across all first-party software projects
- **Scientific literature** structured by citation graphs and domain ontologies
- **Legal codes** structured by the hierarchical organization of statutes, regulations, and case law
- **Biological structure** structured by PDB file format and pathway reaction schemas
- **Personal knowledge** structured by diary timelines, memory graphs, and biographical records
- **Cultural and religious texts** structured by the canonical book/chapter/verse hierarchies of scripture and the formal apparatus of commentary traditions

Coupled with the largest available language model and maximum context window, a query to the Corpus Scientia retrieves the most relevant subgraph across *all* registered knowledge, passes it to the model with full provenance, and receives a synthesis that is grounded, traceable, and verifiably correct at the knowledge layer. This is not retrieval-augmented generation; it is *knowledge-compiler-augmented synthesis*.

The architecture is ready. The adapter protocol requires five methods per new domain. The orchestrator, registry, CLI, MCP server, corpus, and person abstractions require no changes as new domains are added. The path to the Corpus Scientia is, in engineering terms, incremental: one domain compiler at a time, each following the same pattern, each extending the unified query surface without modifying the federation layer.

12 Snapshot System and Differential Queries

Knowledge graphs change as their source corpora evolve. The snapshot system captures `SnapshotMetrics` at each build or ingestion run, keyed by the git tree hash produced by `git write-tree`. A `Snapshot` record carries the tree hash, branch name, ISO 8601 timestamp, module version, a `SnapshotMetrics` struct (node counts, edge counts, coverage score, issue count), and `SnapshotDelta` fields comparing against the immediately preceding and baseline snapshots.

The snapshot history provides a longitudinal record of graph evolution. More importantly, it enables a class of queries that is impossible with a conventional vector index: *differential queries* over the knowledge graph.

Consider two questions that are straightforward to answer once snapshots exist but are unanswerable without them:

1. *How has the Pepys corpus changed since last week?*
   Compare the two nearest snapshots by tree hash. The delta carries exact node and edge counts added and removed, the set of new topic labels that entered the corpus, and the change in semantic coverage score. This is `pycodekg snapshot diff` (or `dockg snapshot diff`) operating on two snapshot keys; each backend CLI exposes this command directly.
2. *What new ideas have entered the Pepys corpus this month?*
   Execute a federated query against both snapshots, compute the set difference of the top-k results, and the remainder is the new knowledge. New nodes that did not exist in the earlier snapshot and rank highly in the current one represent ideas that have entered the compiled corpus. This is *differential federated search*: the same query, two compiled states, a structural diff.

These capabilities are direct consequences of the compilation model. A raw vector index can be updated incrementally, but its history is not addressable; there is no mechanism to query "the index as it was on Tuesday." The compiled knowledge graph, stored in SQLite and keyed by git tree hash, is fully addressable at any point in its history.

12.1 Incremental Ingestion with Temporal Diversity

Large corpora may not be fully ingestable in a single run. KGRAG addresses this with a diversity-preserving batch selection algorithm. When `batch_size` is set, the ingestion pipeline clusters all available entries in a normalised feature space encoding temporal position, content length, lexical density, and named-entity profile, then selects the centroid-nearest entry from each cluster. A batch of 200 entries from a 10,000-entry diary corpus will span the full temporal range and all discoverable themes of that corpus, not merely its beginning.

A state manager tracks processed entry indices. Subsequent runs with `-resume` load this state, skip already-ingested entries, and select a fresh diverse batch from the remainder. The corpus is queryable throughout: an index built from 200 representative entries already covers the semantic space of the full corpus at reduced density. Density increases with each run until full ingestion is achieved.

13 MCP Integration

An MCP server wraps the `KGRAG` orchestrator and exposes a complete tool surface to any MCP-compatible AI agent. Tools are organized into three groups.

Registry tools:

Tool	Function
`kgrag_stats()`	Registry summary: KG count, kinds, built status
`kgrag_list([kind])`	List registered KG entries, optionally filtered
`kgrag_info(name)`	Full detail for a single KG entry
`kgrag_query(q, [k, kinds])`	Federated semantic query, JSON result
`kgrag_pack(q, [k, kinds])`	Federated snippet pack, Markdown output

Corpus tools (`kgrag_corpus_list`, `kgrag_corpus_info`, `kgrag_corpus_create`, `kgrag_corpus_delete`, `kgrag_corpus_add`, `kgrag_corpus_remove`, `kgrag_corpus_query`, `kgrag_corpus_pack`): create and manage named collections of KG instances and execute corpus-scoped federated queries.

Person tools (`kgrag_person_list`, `kgrag_person_info`, `kgrag_person_create`, `kgrag_person_delete`, `kgrag_person_add`, `kgrag_person_remove`, `kgrag_person_update`,

`kgrag_person_query`, `kgrag_person_pack`): create and manage person corpus entries with personal metadata and execute person-scoped federated queries.

The MCP transport is `stdio`, making it compatible with Claude Code, Kilo Code, GitHub Copilot, Cline, and Claude Desktop without additional networking configuration.

14 Comparison with Inference-Based Graph Construction

GraphRAG [1] is the most widely deployed system in the vicinity of this work. Both KGRAG and GraphRAG augment retrieval with a knowledge graph; the origin and guarantees of that graph differ substantially.

In GraphRAG, a language model extracts entities and relationships from text documents. The graph is a product of language model inference; its correctness is bounded by extraction prompt quality and model comprehension. Errors introduced during extraction are silent: the system has no mechanism to verify that an extracted relationship corresponds to anything in the source.

In KGRAG, knowledge graphs are derived from formal artifacts by deterministic programs. A `CALLS` edge exists because a call expression was found in the AST at the recorded line number. A metabolic reaction edge exists because it appears in the source schema. A statutory cross-reference edge exists because a citation was found in the formal text. No edge is inferred; every edge is verified against a source address.

Table 1: Inference-based vs. structure-derived knowledge graph retrieval

Property	GraphRAG [1]	KGRAG
Graph source	LLM extraction from text	Formal source parsing
Graph correctness	Probabilistic	Deterministic
Edge evidence	NL relationship description	Source address + line
LLM required to build	Yes	No
LLM required to query	No	No
Domain generality	Text corpora	Any formally structured domain
Hallucination risk (KG layer)	Present	Zero
Hallucination risk (synthesis)	Present	Present (bounded)
Result provenance	Document reference	Node ID + file + line
Query latency	Seconds to minutes	Under one second (local)

The two systems are not in competition for the same problem. GraphRAG is designed for global sense-making over large unstructured text corpora. KGRAG is designed for structural navigation of formally specified artifacts, with a zero-hallucination guarantee at the knowledge layer.

15 Conclusion

KGRAG demonstrates that structural knowledge from heterogeneous source domains can be federated under a single uniform interface without collapsing domain specificity into a common text representation. The adapter protocol cleanly separates domain-specific extraction from domain-agnostic orchestration.

The central findings of this work are:

1. **The Ontology Principle.** The power of structural KG retrieval scales with the formal completeness of the source domain's ontology. Programming languages are the ideal case; biochemical pathways, legal codes, protein structure files, and scripture hierarchies follow the same pattern. Where formal structure exists, the derived graph is correct by construction.
2. **The Knowledge Compiler.** KGRAG is not a retrieval system that happens to use graphs. It is a knowledge compiler: it transforms formally structured source artifacts into relational representations

that preserve and expose the structure of the originals. The compiled knowledge graph *is* the corpus, represented for navigation.

3. **Universal Federation.** Because all domain adapters expose the same five-method interface, all compiled knowledge is queryable through one interface. The scientist, lawyer, software engineer, and biographer query the same system. One command, one ranking, full provenance.
4. **Zero Hallucination at the Knowledge Layer.** When KGRAG output is passed to a language model for synthesis, the model receives verified facts with source addresses. Hallucination risk is bounded to the synthesis stage alone and is zero at the knowledge-retrieval stage. This is an unconditional guarantee: it holds regardless of model scale, prompt engineering, or retrieval quality.
5. **The Path to the Corpus Scientia.** The *TreeOfKnowledge™* is achievable. Each new domain requires five adapter methods. The federation layer requires no changes. The architecture is ready; the remaining work is building the domain compilers.

The result is a retrieval system that is fast, auditable, and correct by construction at the structural layer. When coupled with the largest available language model for synthesis, it provides grounded context whose every element can be verified against the source. The correctness guarantee holds regardless of the language model's accuracy, because the model receives facts rather than constructs them. That is the ultimate power of the design.

References

[1] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. `https://arxiv.org/abs/2404.16130`

[2] Eric G. Suchanek. PyCodeKG: Deterministic code navigation via knowledge graphs, semantic indexing, and grounded context packing. Technical report, Flux-Frontiers, 2025. `https://github.com/Flux-Frontiers/pycode_kg`

[3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP 2019*, 2019. `https://arxiv.org/abs/1908.10084`